



PRÉ-RENTRÉE 2026-2027

Stage d'entrée en Première NSI

Maîtrise opérationnelle de Python

5 séances de 2 heures - 10 heures

Python 3 - théorie, pratique, tests et projet

Groupe : Ahmad BELDI et Ahmed BENHADJ SALEM

1. Positionnement et décision pédagogique

1.1 Finalité réelle du stage

Le stage est **centré sur la maîtrise opérationnelle de Python**, car la NSI n'a pas été enseignée auparavant dans le parcours scolaire ordinaire d'Ahmad BELDI. Il ne s'agit pas de reproduire en dix heures l'intégralité d'une année de Première NSI. La cible réaliste et exigeante est la suivante :

rendre disponibles les constructions Python et les méthodes de programmation nécessaires pour suivre la Première NSI sans blocage, comprendre un programme, en écrire un, le décomposer, le tester, le corriger et expliquer son fonctionnement.

Le programme officiel de Première NSI en vigueur reste celui du BO spécial n° 1 du 22 janvier 2019. Il privilégie Python 3, sans faire de l'expertise dans un langage une fin en soi. Il attend notamment les séquences, affectations, conditions, boucles, fonctions, spécifications, assertions, jeux de tests, types construits, tables de données et algorithmes classiques.

1.2 Limite assumée

Les thèmes Web, réseaux, architecture et systèmes figurent au programme annuel, mais ne constituent pas l'objet principal de ce stage. Ils apparaissent seulement dans les rituels, le mémento et la feuille de route annuelle. La totalité des dix heures est structurée autour de Python et de l'algorithmique.

1.3 Effectif et profils

Élève	Situation	Données disponibles	Décision pédagogique
Ahmad BELDI	Passage de Seconde en Première ; aucune NSI formellement enseignée	13/18 questions traitées ; programmation 22,2 % ; représentation binaire, Web et données en tables à consolider ; réseaux à situer	Parcours Fondations guidées : modèle mental de l'exécution, traçage, syntaxe, indexation, fonctions et tests

Élève	Situation	Données disponibles	Décision pédagogique
Ahmed BENHADJ SALEM	Candidat individuel ; a déjà présenté une première partie sans réussite	Test Première : 18/18 traitées, quatre domaines solides, programmation 55,6 % ; test Terminale : six domaines solides mais erreur d'accumulateur	Parcours Fiabilisation et autonomie : corriger les automatismes fragiles, coder sous contrainte, tester, documenter et expliquer

1.4 Conceptions prioritaires

Ahmad BELDI

- pense qu'une affectation conserve un lien dynamique entre deux variables ;
- interprète `x = x + 1` comme un nom de variable ;
- oublie le cas d'égalité dans la négation d'une condition ;
- interprète `range(3)` comme une seule exécution ;
- commence l'indexation d'une liste à 1 ;
- ne distingue pas la longueur d'une liste du dernier indice ;
- confond paramètre, valeur renvoyée et portée locale ;
- ne connaît pas le retour implicite `None`.

Ahmed BENHADJ SALEM

- maîtrise une grande partie des repères théoriques, mais conserve des erreurs fortement assurées ;
- confond parfois la négation logique ;
- inverse l'usage de `for` et `while` ;
- confond `len(L)` et l'indice du dernier élément ;
- ne stabilise pas encore le retour `None` ;
- a commis une erreur d'accumulation en confondant nombre d'itérations et somme produite ;
- doit transformer une connaissance déclarative en code fiable, testé et explicable.

2. Référentiel Python de Première NSI

2.1 Matrice de couverture

Capacité attendue pendant l'année	Couverture dans le stage	Séance	Niveau visé à la sortie
Séquences, affectations, expressions et types	Complète	1	Autonome

Capacité attendue pendant l'année	Couverture dans le stage	Séance	Niveau visé à la sortie
Booléens, comparaisons, <code>and</code> , <code>or</code> , <code>not</code> , conditions	Complète	1	Autonome
Boucles bornées <code>for</code> , <code>range</code>	Complète	2	Autonome
Boucles non bornées <code>while</code> , terminaison	Complète	2	Autonome sur cas simples
Compteurs, accumulateurs, recherche de minimum/maximum	Complète	2	Autonome
Fonctions, paramètres, arguments, <code>return</code> , <code>None</code> , portée	Complète	3	Autonome
Préconditions, postconditions, assertions, tests	Complète	3	Autonome sur fonctions simples
Documentation et lecture de documentation	Complète	3	Utilisable
Chaînes, tuples, listes, tableaux, compréhensions	Complète	4	Autonome
Dictionnaires, clés/valeurs/items	Complète	4	Autonome
Tableaux de tableaux	Introduite	4	Guidé
Import CSV, sélection, tri, cohérence, doublons	Complète	4-5	Autonome sur un fichier simple
Recherche séquentielle, extremum, moyenne	Complète	2 et 5	Autonome
Tri par insertion et par sélection	Un algorithme écrit ; l'autre analysé	5	Guidé puis transférable
Recherche dichotomique	Introduite et tracée	5	Compréhension et adaptation
Coût linéaire, quadratique, logarithmique	Intuition structurée	5	Comparaison qualitative
Invariant et variant de boucle	Initiation	2 et 5	Savoir les expliquer sur un exemple
Bibliothèques Python	Utilisation standard de <code>csv</code> et de <code>math</code>	3-5	Lire une documentation simple
K plus proches voisins et glouton	Panorama et feuille de route	5	Non évalué au stage

2.2 Compétences transversales

- décomposer un problème en sous-problèmes ;
- écrire un algorithme avant le code lorsque la tâche est complexe ;
- tracer manuellement un programme ;
- expliquer le rôle de chaque variable ;
- spécifier les entrées et sorties ;
- choisir des tests normaux, limites et invalides ;
- corriger un bug à partir d'une observation ;
- lire et réutiliser du code existant ;
- présenter oralement une solution.

3. Objectifs de sortie

À l'issue des cinq séances, l'élève doit pouvoir :

1. lire un programme court et produire une table de trace exacte ;
2. distinguer affectation, comparaison et calcul ;
3. manipuler les types `int`, `float`, `str`, `bool` et `None` ;
4. écrire une condition correcte et sa négation ;
5. choisir entre `for` et `while` ;
6. utiliser correctement `range`, un compteur et un accumulateur ;
7. écrire une fonction avec paramètres et valeur renvoyée ;
8. distinguer `print` et `return` ;
9. écrire une docstring, une précondition et des assertions de test ;
10. manipuler listes, tuples et dictionnaires ;
11. parcourir, filtrer et transformer une table CSV ;
12. écrire une recherche séquentielle et une recherche d'extremum ;
13. comprendre les principes d'un tri et d'une recherche dichotomique ;
14. produire un petit programme découpé en fonctions, documenté et testé ;
15. expliquer ses choix et contrôler son résultat.

4. Progression des cinq séances

Séance	Axe	Théorie	Pratique principale	Livable
1	Variables, types, booléens et conditions	État mémoire, affectation, expression, type, logique	Classificateur de mesure et table de trace	Fiche « lire et écrire un programme simple »

Séance	Axe	Théorie	Pratique principale	Livrable
2	Boucles et schémas algorithmiques	<code>for</code> , <code>while</code> , compteur, accumulateur, invariant, terminaison	Analyse d'une série de mesures	Bibliothèque de parcours de listes
3	Fonctions, contrats, tests et débogage	Paramètre, argument, retour, portée, <code>None</code> , assertions	Module de fonctions testées	Gabarit de fonction propre et testée
4	Listes, tuples, dictionnaires et CSV	Mutabilité, indexation, compréhensions, enregistrements	Import et filtrage de <code>mesures_capteurs.csv</code>	Analyseur de table version 1
5	Algorithmes et mini-projet	Recherche, tri, dichotomie, coût	Projet final d'analyse de données	Programme documenté + bilan individuel

5. Déroulés synthétiques

Séance 1 - Variables, types, booléens et conditions

- 0-10 min : installation, règles de nommage et test de l'environnement ;
- 10-25 min : table de trace sur les affectations ;
- 25-45 min : types, opérateurs et conversions ;
- 45-65 min : expressions booléennes, conditions et négations ;
- 65-70 min : pause ;
- 70-95 min : programmation guidée ;
- 95-110 min : parcours différenciés ;
- 110-118 min : tests et revue croisée ;
- 118-120 min : exit ticket.

Séance 2 - Boucles et schémas algorithmiques

- 0-10 min : rituel de traçage ;
- 10-30 min : `range`, bornes et indices ;
- 30-50 min : compteurs et accumulateurs ;
- 50-65 min : choix `for/while`, terminaison ;
- 65-70 min : pause ;
- 70-100 min : atelier « analyse d'une série » ;
- 100-112 min : différenciation ;
- 112-120 min : invariant, contrôle et exit ticket.

Séance 3 - Fonctions, contrats et tests

- 0-10 min : rituel ;
- 10-30 min : paramètres, arguments et portée ;
- 30-48 min : `return`, `print` et `None` ;
- 48-65 min : docstrings, préconditions et postconditions ;
- 65-70 min : pause ;
- 70-100 min : écriture d'un module de fonctions ;
- 100-112 min : jeux de tests et débogage ;
- 112-120 min : revue de code et exit ticket.

Séance 4 - Structures de données et tables

- 0-10 min : rituel ;
- 10-28 min : chaînes, tuples, listes et indexation ;
- 28-45 min : mutation, alias et copie ;
- 45-65 min : dictionnaires et enregistrements ;
- 65-70 min : pause ;
- 70-100 min : lecture CSV et filtrage ;
- 100-112 min : tri, doublons et cohérence ;
- 112-120 min : synthèse et exit ticket.

Séance 5 - Algorithmes et projet final

- 0-10 min : quiz cumulatif ;
- 10-27 min : recherche séquentielle et extremum ;
- 27-43 min : tri par insertion ;
- 43-58 min : recherche dichotomique et coûts ;
- 58-63 min : présentation du projet ;
- 63-68 min : pause ;
- 68-105 min : réalisation du projet final ;
- 105-115 min : démonstration orale et tests ;
- 115-120 min : bilan et plan de travail.

6. Différenciation des deux élèves

6.1 Tronc commun

Le tronc commun représente environ 70 % du temps. Les deux élèves travaillent les mêmes notions et le même projet afin de favoriser les explications croisées.

6.2 Ahmad BELDI - parcours Fondations guidées

- tables de trace obligatoires avant exécution ;
- code à trous puis code autonome ;
- fonctions courtes, une difficulté à la fois ;
- aide graduée sur `range`, indices, `len`, paramètres et `return` ;
- tests fournis au départ, puis à compléter ;
- projet final avec squelette détaillé.

Indicateurs

- 4 tables de trace exactes sur 5 ;
- aucune confusion affectation/comparaison ;
- `range` et indexation maîtrisés ;
- fonction avec paramètre et retour correctement écrite ;
- liste parcourue sans erreur de borne ;
- projet exécuté sans erreur et expliqué.

6.3 Ahmed BENHADJ SALEM - parcours Fiabilisation et autonomie

- même tronc commun, mais moins d'étapes fournies ;
- cas limites et entrées invalides ;
- obligation de rédiger les tests avant certaines fonctions ;
- tâches chronométrées courtes ;
- comparaison de deux algorithmes ;
- projet final avec spécification, sans squelette détaillé ;
- oral de justification inspiré des exigences pratiques du baccalauréat.

Indicateurs

- négations logiques correctes ;
- choix `for/while` justifié ;
- accumulateur fiable ;
- distinction `len(L)` / dernier indice ;
- retour `None` compris ;
- tests incluant cas limite ;
- projet final documenté et soutenu sans dépendance forte à l'enseignant.

7. Évaluation

7.1 Avant

- test initial NSI déjà administré ;
- mini-diagnostic pratique de 25 minutes ;
- entretien oral de cinq minutes par élève.

7.2 Pendant

- cinq rituels ;
- cinq exit tickets ;
- journal des aides ;
- validation progressive des fichiers Python ;
- revue croisée des tests.

7.3 Finale

- quiz théorique court ;
- tâche pratique sur ordinateur ;
- mini-projet ;
- présentation orale de deux minutes ;
- grille de compétences, sans note de classement.

8. Feuille de route après le stage

Le stage couvre le socle Python. La poursuite de Première devra réactiver puis approfondir :

- binaire, hexadécimal, complément à deux, flottants et encodage ;
- listes de listes, dictionnaires et tables ;
- tris, dichotomie, k plus proches voisins et algorithmes gloutons ;
- Web, HTTP, formulaires et client-serveur ;
- architecture de von Neumann, réseau et système d'exploitation ;
- projets en binôme avec documentation et tests.

9. Références officielles

- Ministère de l'Éducation nationale, **Programme de numérique et sciences informatiques de première générale**, BO spécial n° 1 du 22 janvier 2019, NOR MENE1901633A.
- Éduscol, **Programmes et ressources en numérique et sciences informatiques - voie générale**, mise à jour février 2026.
- Les ressources Éduscol sur les types construits, les tables de données, la mise au point de programmes testés et la recherche dichotomique ont servi de contrôle de cohérence.